09강

프로그래밍 언어론

타입

컴퓨터과학과 이관용 교수

학습목차



1 타입의 개요

2 타입의 분류

3 단순타입

01

타입의 개요

타입

타입 type

- 데이터 집합과 연산 집합의 결합

- 데이터 집합
  - 처리 대상이 되는 데이터들의 집합
  - 예: 정수형의 데이터 집합 = { …, −2, −1, 0, 1, 2, … }
- 연산 집합
  - 해당 데이터에 적용 가능한 연산의 집합
  - 예: 정수형의 연산 집합 = { +, −, *, /, %, … }

타입

- 변수의 속성 중 하나
  - 해당 변수에는 그 타입이 지원하는 데이터와 연산만 적용할 수 있음
- 서브프로그램의 인수와 반환에도 이용

```
int swap (int x, int y)
```

- 타입 안전성
  - 연산/서브프로그램의 적용 전후 타입의 일치가 중요 → 정확한 연산 결과를 보장
  - 프로그램은 "타입 안전 type safe하다"라는 표현
    - 프로그램 내 모든 연산 및 함수에 대해서 다음 성질을 만족하는 경우
      함수 f의 타입이 $f(x)$: $A \rightarrow B$라면 모든 $a \in A$에 대해 $f(a) \in B$이어야 함

    → "타입 오류가 발생하지 않는다"

타입 안전성에 따른 프로그래밍 언어의 구분

강타입 언어 strongly typed

- 모든 타입 오류를 검출해내는 언어
- Haskell, ML, Java (타입 캐스팅 제외하는 경우)

약타입 언어 weakly typed

- 타입 오류를 검출, but 일부 타입 오류를 허용하는 언어
- C → 공용체, 타입 캐스팅을 통해 타입 검사 회피 가능

무타입 언어 typeless

- 타입 선언문이 없고 어떤 대상의 타입이 계속 바뀔 수 있는 언어
- Python 등 대부분의 스크립트 언어

02

타입의 분류

타입의 분류 방법

- 데이터의 형태에 따라
  - 정수형, 실수형, 문자형, 배열, 구조체 등
- 타입 정의에 사용자의 개입 여부에 따라
  - 원시타입 primitive type
  - 사용자정의타입 user-defined type
- 데이터 요소의 형태에 따라
  - 단순타입 simple type
  - 복합타입 composite type

사용자 개입 여부에 따른 분류

- 원시타입 vs 사용자정의타입
  - 프로그래밍 언어에서 기본적으로 제공하는 타입
  - "미리 정의된 타입 predefined type", "내장타입 built-in type"
  - 대부분의 언어 → 정수형, 실수형, 문자형, 논리형 등 제공
    - C, C++ → `int`, `float`, `char`, 배열 등
    - Java → `int`, `float`, `char`, `boolean`, 배열 등

사용자 개입 여부에 따른 분류

- 사용자정의타입 vs 원시타입
  - 사용자가 직접 정의해서 사용하는 타입
  - 배열 (크기와 데이터 종류 지정), 구조체 (데이터 종류 지정)
    - C → `enum`, `struct`, `union` 등
    - C++, Java → `enum`, `class` 등

데이터 요소의 형태에 따른 분류

단순타입 vs 복합타입

- 데이터 집합의 요소가 하나의 데이터로만 구성된 경우
- "스칼라타입 scalar type", "원자타입 atomic type"
- 정수형, 실수형, 문자형, 논리형, 열거형 등
  - C++ → `int`, `float`, `char`, `bool`, `enum` 등

데이터 요소의 형태에 따른 분류

- 복합타입 vs 단순타입
  - 데이터 집합의 요소가 데이터들의 구조로 구성된 경우
    - 단순타입 또는 다른 복합타입을 활용하여 요소를 구성
  - "구조타입 structured type"
  - 배열, 구조체, 클래스 등
    - C → 배열, `struct`, `union` 등
    - C++, Java → 배열, `class` 등

03

단순타입

정수형 integer type

- 정수 데이터를 다루는 타입

- 데이터 집합
  - $\{-\infty, \cdots, -2, -1, 0, +1, +2, \cdots, +\infty\}$ 중 일부만 표현
    - 표현 가능한 정수 범위는 할당된 메모리 크기에 의존
  - 표현 가능한 정수 범위에 따라 다양한 타입이 존재
    - C → `short`, `int`, `long`, `long long`,
      `unsigned short`, `unsigned int`, `unsigned long`, `unsigned long long`
    - Java → `byte`, `short`, `int`, `long`

정수형_C, C++

타입명	최소 사용 비트	최소 데이터 집합 범위
short	16	$-2^{15} \sim 2^{15}-1$ (-32,768 ~ 32,767)
int	16	$-2^{15} \sim 2^{15}-1$
long	32	$-2^{31} \sim 2^{31}-1$ (-2,147,483,648 ~ 2,147,483,647)
long long	64	$-2^{63} \sim 2^{63}-1$

타입명	최소 사용 비트	최소 데이터 집합 범위
unsigned short	16	$0 \sim 2^{16}-1$ (0 ~ 65,535)
unsigned int	16	$0 \sim 2^{16}-1$
unsigned long	32	$0 \sim 2^{32}-1$
unsigned long long	64	$0 \sim 2^{64}-1$

정수형_Java

타입명	사용 비트	데이터 집합 범위
byte	8	$-2^7 \sim 2^7-1$
short	16	$-2^{15} \sim 2^{15}-1$
int	32	$-2^{31} \sim 2^{31}-1$
long	64	$-2^{63} \sim 2^{63}-1$

- C/C++의 정수형과의 차이점
  - 각 타입의 사용 비트가 고정되어 있음
  - unsigned가 붙는 정수형이 없음

정수형

연산 집합

- 사칙 연산 → 덧셈, 뺄셈, 곱셈, 나눗셈
- 연산의 결과 값 → 언제나 범위 내의 정수
  - 10 / 3 → 3
  - C/C++에서 16비트로 구현된 `short` 타입의 경우 (-32,768 ~ 32,767)

    32767 + 1 → 최댓값 + 1 → 최솟값 (-32,768)

    -32768 -1 → 최솟값 - 1 → 최댓값 (32,767)

정수형

연산 집합

- 나머지 연산 → mod (Pascal), % (C,C++,Java)
  - 9 % 5 → 4
- 비트 연산 → ^, &, | 등
- 관계 연산 → <, >, <=, >=, ==, != 등
  - 관계 연산의 결과값 → 논리형

실수형 real type

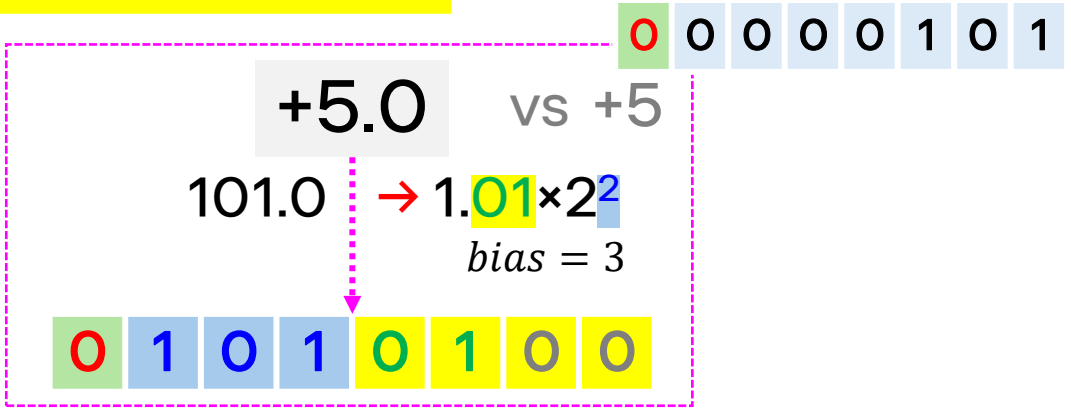
- 실수 데이터를 다루는 타입
  - -100.05, -8.7, 0.0, 1.234, 256.3500187 등

- 데이터 집합
  - 전체 실수 중 사용 비트의 크기에 따라 일부 범위의 실수만 표현 가능
  - 실수 데이터 → "근사값"
  - 일반적으로 부동 소수점 수 floating-point number로 표현

실수형

- 부동소수점 수 $\pm$가수$\times 2^{\text{지수}}$



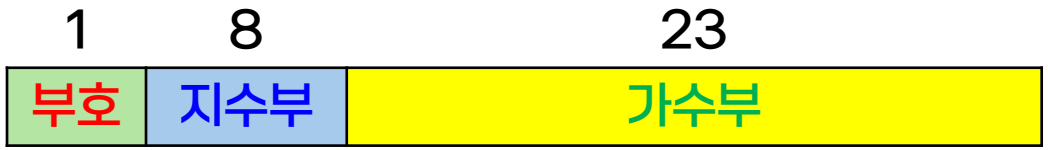


- 부호
  - 양수 → 0, 음수 → 1
- 지수부 → '지수 + bias'
  - bias → 지수부의 크기에 따라 결정 → 지수부가 m비트이면 $bias=2^{m-1}-1$
- 가수부 → 정규화 이후 소수점 이하 부분만 표현
  - 정규화 → 실수를 1.xxxx 형태를 갖도록 변환

실수형

C, C++, Java

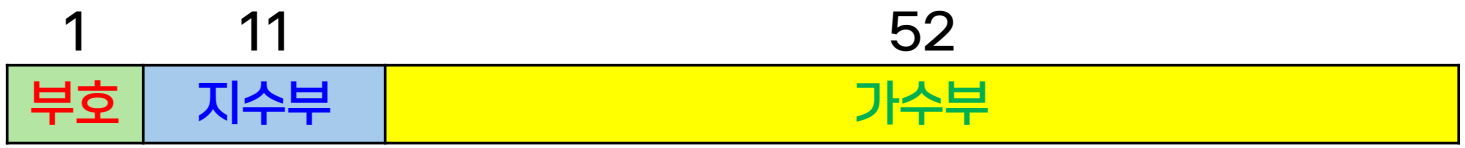
- `float` → 32비트 → 약 -3.403×10^{38} ~ 약 -1.175×10^{-38}, 0, 약 1.175×10^{-38} ~ 약 3.403×10^{38}



$bias = 127$

→ -126 ~ 127

- `double` → 64비트 → 약 -1.798×10^{308} ~ 약 -2.225×10^{-308}, 0, 약 2.225×10^{-308} ~ 약 1.798×10^{308}



$bias = 1023$

→ -1022 ~ 1023

실수형

연산 집합

- 사칙 연산 → 덧셈, 뺄셈, 곱셈, 나눗셈
- 연산의 결과 값 → 실수형
  - 1.5 + 2.5 → 4.0
  - 최댓값/최솟값을 초과는 경우 → 무한대(`inf`/`-inf`)로 표현

    5E37× 10.0 → 최댓값(약 3.403×10^{38}) 초과 → 무한대 `inf`
- 관계 연산 → <, >, <=, >=, ==, != 등
  - 관계 연산의 결과 값 → 논리형

문자형 character type

- 하나의 문자 데이터를 다루는 타입
  - 'A', 'a', '#', '.', '3' 등
- 데이터 집합
  - 문자 코드 체계에서 사용할 수 있는 문자들의 집합
  - ASCII → 특수 기호, 구두점, 숫자, 영어 대소문자 등 128개의 문자로 구성
    - C, C++ → char → 8비트
  - 유니코드 → 전 세계의 다양한 문자 표현 가능
    - Java → char → 16비트

문자형

- 연산 집합
  - 관계 연산
    - 연산의 결과 값 → 논리형
    - 'A' < 'B' → true
  - C, C++ → char 타입을 8비트 정수형으로 간주
    → 사칙 연산, 비트 연산 등이 가능
    - 'A' + 32 → 65 + 32 → 'a'
    - 'a' - 32 → 97 - 32 → 'A'

논리형 logical type

- 참과 거짓의 논리 데이터를 다루는 타입
- 데이터 집합
  - 참(true), 거짓(false)

언어	타입명	데이터 집합
C	`_Bool`	{1, 0}
C++	`bool`	{true, false}
Java	`boolean`	{true, false}

- 연산 집합
  - 논리곱(and), 논리합(or), 논리부정(not) 등
    - 연산의 결과 값 → 논리형
    - 예: `true && false` → `false`

열거형 enumeration type

- 순서 관계가 있는 이름들을 데이터로 다루는 타입

- 데이터 집합
  - 사용자가 직접 지정한 이름들
  - 각 이름은 0 이상의 정수와 대응 → 이름 간의 순서 관계가 성립됨

열거형

C, C++ → enum 타입

- 각 이름은 0부터 순서대로 대응되는 경우

```
enum Year { Jan, Feb, Mar, Apr, May, Jun, Jul,
            Aug, Sep, Oct, Nov, Dec };
```

- 사용자가 각 이름에 대응되는 정수를 직접 지정하는 경우

```
enum Year { Jan=1, Feb, Mar, Apr=10, May=15,
            Jun=1, Jul, Aug, Sep, Oct, Nov, Dec };
```

동일한 순서를 가짐

열거형

연산 집합

- 관계 연산
  - 연산의 결과 값 → 논리형
- C → 대응되는 정수 값을 기반으로 사칙 연산도 가능

```
#include <stdio.h>
int main()
{
  enum Year { Jan, Feb, Mar, Apr, May, Jun,
              Jul, Aug, Sep, Oct, Nov, Dec};
  enum Year go;
  for (go = Jan; go <= Dec; go++)
    printf ("%d\n", go);
  return 0;
}
```

→

```
0
1
2
3
4
5
6
7
8
9
10
11
```

정리하기

타입의 개요

- 정의(데이터 집합 + 연산 집합), 타입 안전성에 따른 PL 분류(강타입 언어, 약타입 언어, 무타입 언어)

타입의 분류

- 데이터 형태 → 정수형, 실수형, 문자형, 배열, 구조체 등
- 사용자의 개입 여부 → 원시타입(PL이 기본적으로 제공) vs 사용자정의타입(사용자가 직접 정의)
- 데이터 요소의 형태 → 단순타입(하나의 데이터로 구성) vs 복합타입(데이터들의 구조로 구성)

단순 타입

- 정수형 → 할당되는 메모리 크기에 따라 다양한 타입 존재, 연산(+, -, *, /, %, 비트, 관계)
- 실수형 → 부동소수점 수(부호+지수부+가수부), 사칙연산, 관계 연산
- 문자형 → 문자 코드체계(ASCII, 유니코드), 관계 연산, C/C++(사칙연산, 비트 연산)
- 논리형 → 데이터 집합={ 참, 거짓}, 논리 연산(AND, OR, NOT 등)
- 열거형 → 순서 관계가 있는 이름들을 데이터로 다루는 타입, 관계 연산, enum 타입

10강

다음시간안내 ▶▶▶

복합타입